

PROGREE

DevOps Internship — Task Completion Report

Tasks 1 – 3: LinkedIn Announcement, Containerization, and CI/CD Pipeline

Intern Name:	Muhammad Usman Ameer
Student ID:	B1/019
Internship Program:	DevOps Internship — Progree
Duration:	5th June 2026 – 4th July 2026
Report Covers:	Task 1, Task 2, Task 3

TASK 1: Professional LinkedIn Announcement

Status: Completed

Objective: Share the official internship acceptance on LinkedIn with a compelling caption, the Progree tag, and required hashtags.

A LinkedIn post was published announcing acceptance into the Progree DevOps internship. The post includes the official Internship Offer Letter, a caption outlining learning goals for the internship (Docker, CI/CD, Terraform, Kubernetes), a direct tag of Progree, and the required hashtags: #devops #docker #cicd #progree #kubernetes #terraform #internship.



Figure 1.1 — Published LinkedIn post with attached official offer letter

TASK 2: Application Containerization & Asset Optimization

Status: Completed

Objective: Build a modular, reliable containerized environment using a multi-stage Dockerfile, minimize the final image footprint, securely map environment variables, and configure functional port routing.

A multi-stage Dockerfile was authored for the application (Node.js backend with PostgreSQL database), orchestrated via docker-compose with two services — backend and db. The stack was built and verified running successfully in Docker Desktop, with environment variables securely injected via .env / compose configuration and explicit container-to-host port mappings.

2.1 Containers Running in Docker Desktop

Both containers (progree-devops backend and the PostgreSQL database) are shown running with live CPU and memory usage, confirming the image builds and starts correctly.

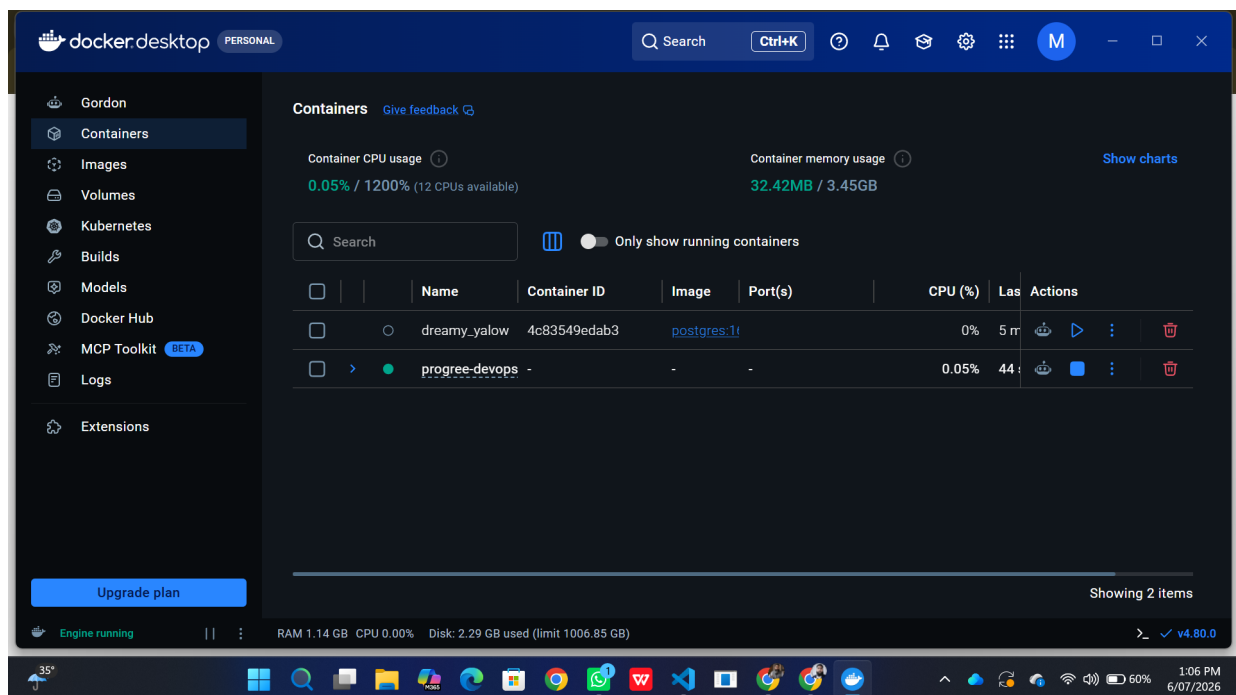


Figure 2.1 — Docker Desktop: Containers view showing progree-devops running

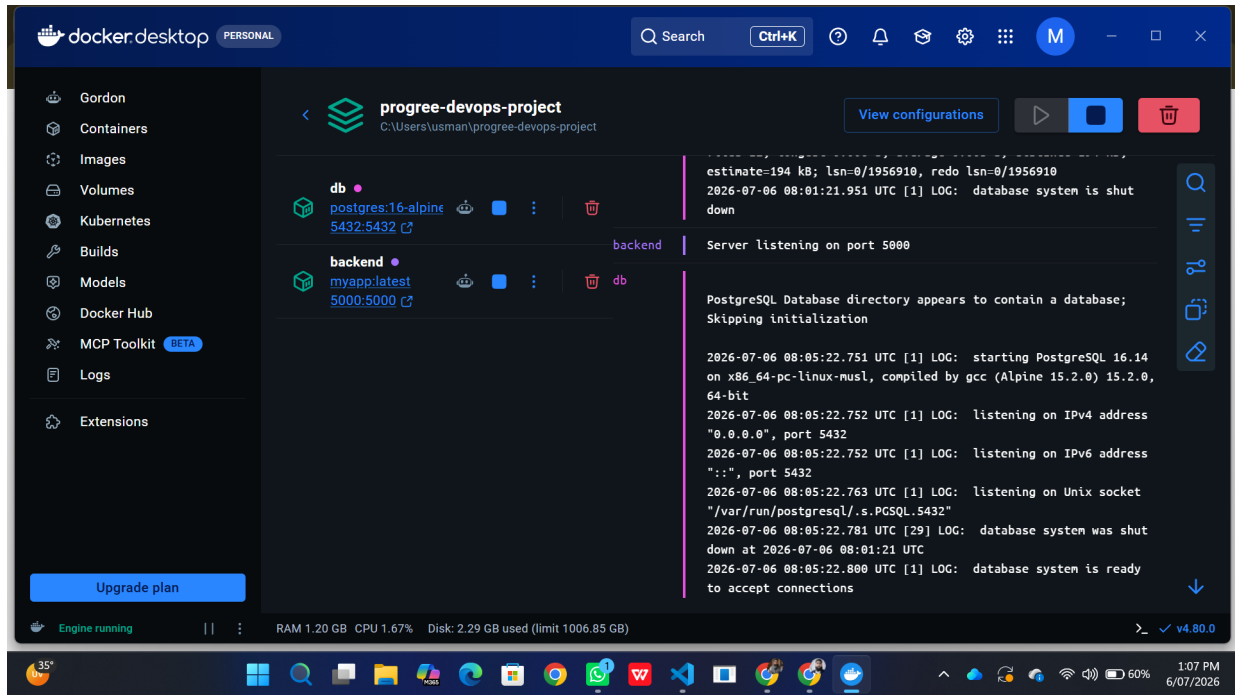


Figure 2.2 — Compose project (progree-devops-project) with backend + db services, port mappings (5432:5432, 5000:5000), and live container logs confirming successful startup

2.2 Application Verified in Browser

The containerized application was accessed via the mapped host port and confirmed to be served correctly by the backend container.



Progree DevOps Demo

This frontend is built during the container image build and served by the Node backend.



Figure 2.3 — Application accessible at localhost:5000, served by the Node backend container

2.3 Port & Connectivity Verification

Network connectivity to the database container port was tested and confirmed successful using PowerShell's Test-NetConnection utility.

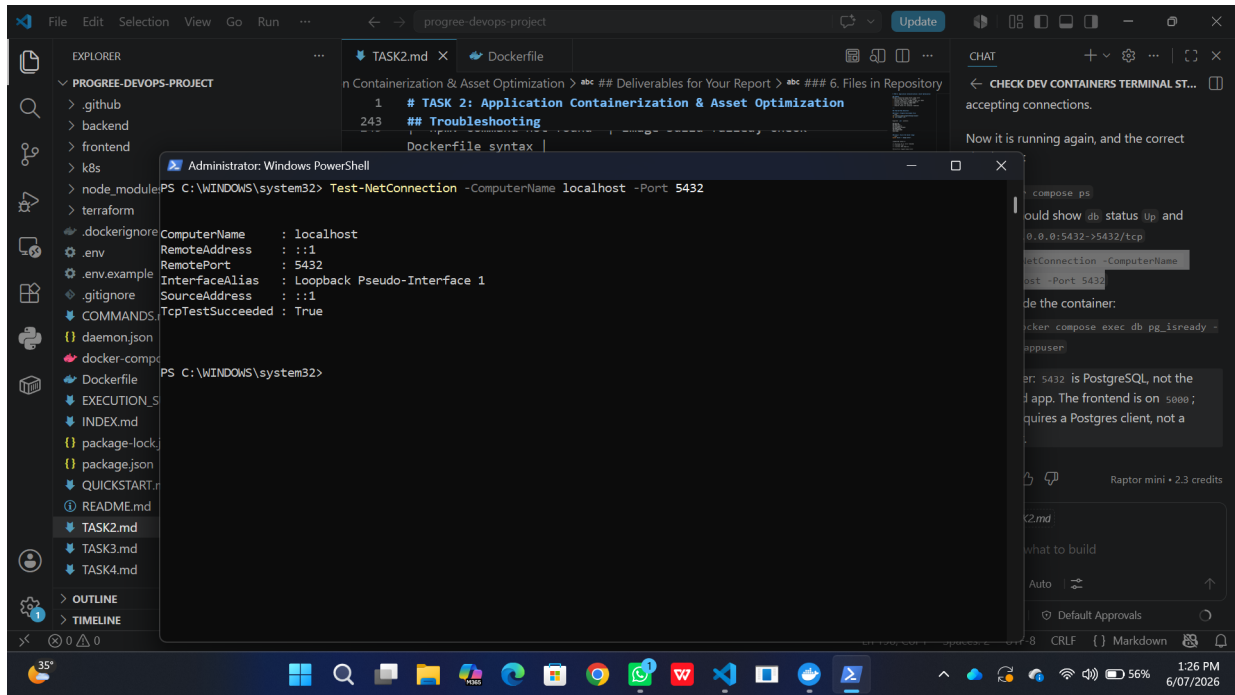


Figure 2.4 — Test-NetConnection confirming TCP connectivity to the PostgreSQL container on port 5432 (TcpTestSucceeded: True)

The backend service's health endpoint was also tested directly, returning HTTP 200 OK, confirming the containerized backend is healthy and responding correctly.

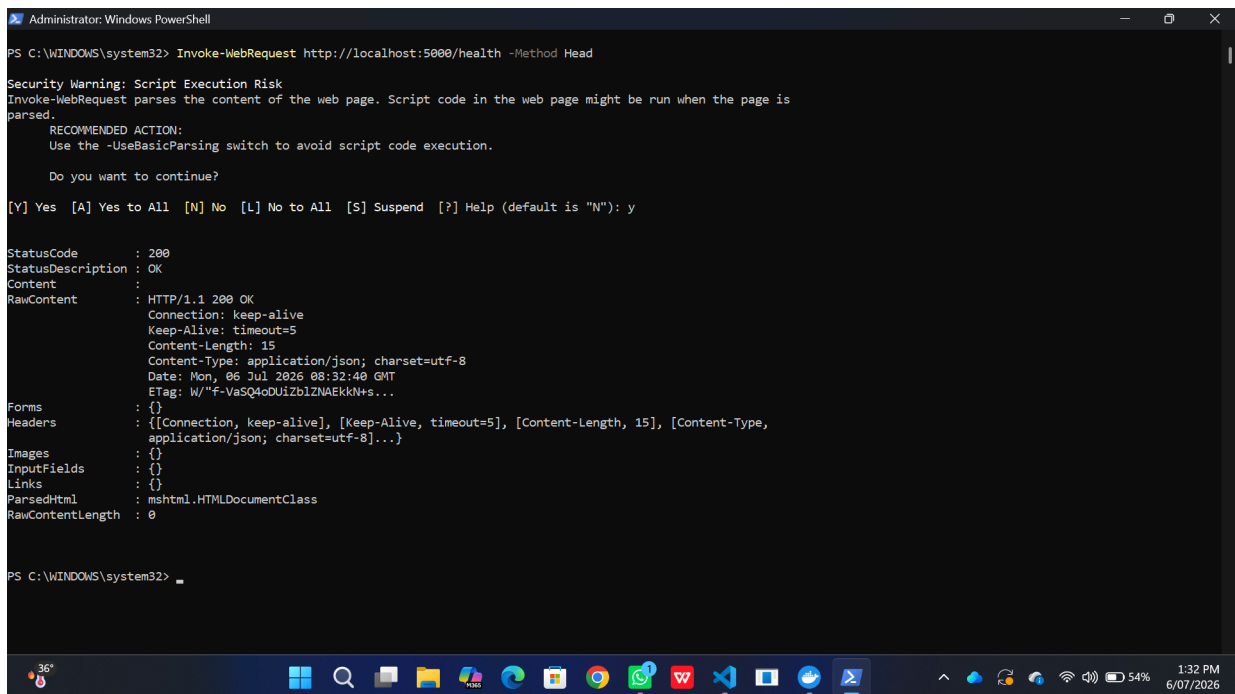


Figure 2.5 — Health check request to `http://localhost:5000/health` returning `StatusCode 200 OK`

2.4 Summary of Port Routing

Service	Container Port	Host Port	Verified Status
---------	----------------	-----------	-----------------

backend	5000	5000	Healthy (200 OK)
db (postgres)	5432	5432	Reachable (TCP test succeeded)

TASK 3: Multi-Stage Automated CI/CD Deployment Pipeline

Status: Completed

Objective: Configure an automated GitHub Actions pipeline that triggers on push, pulls code, lints it, runs unit tests, builds the Docker image, and logs deployment status.

A GitHub Actions workflow (ci-cd.yml) was created in the Progree-DevOps-Internship repository. The pipeline was triggered automatically on a push to the main branch and completed successfully across all four defined stages.

3.1 Workflow Run Overview

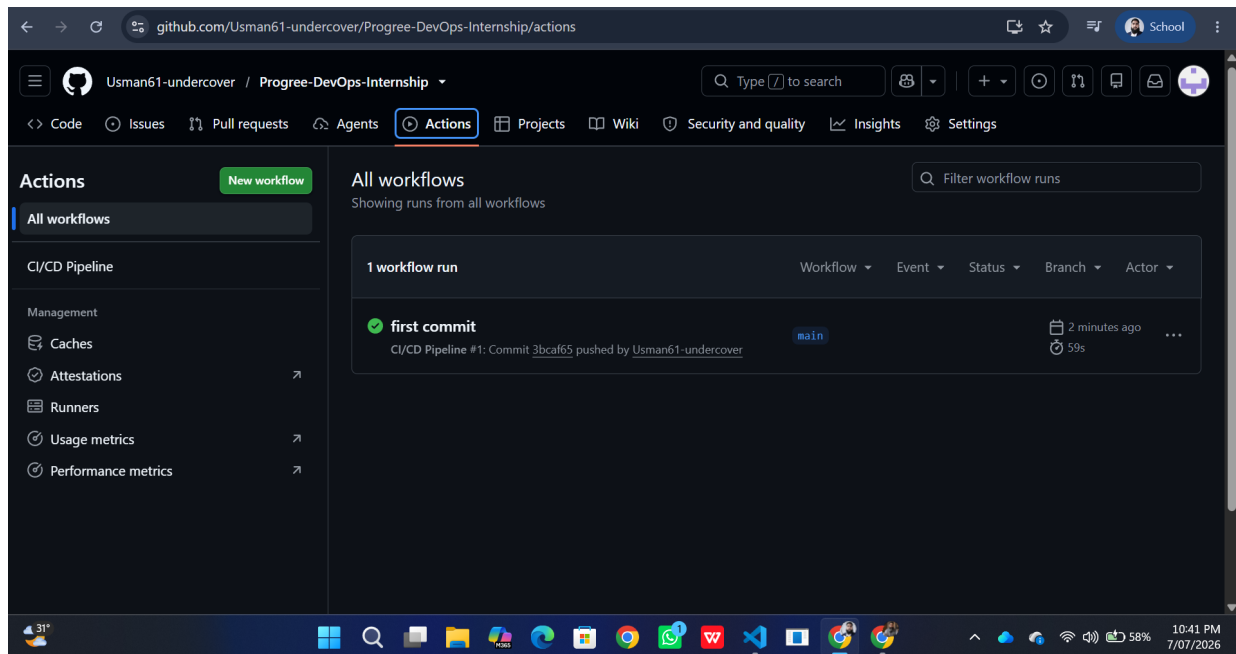


Figure 3.1 — GitHub Actions: workflow run triggered by push, completed successfully in 59s

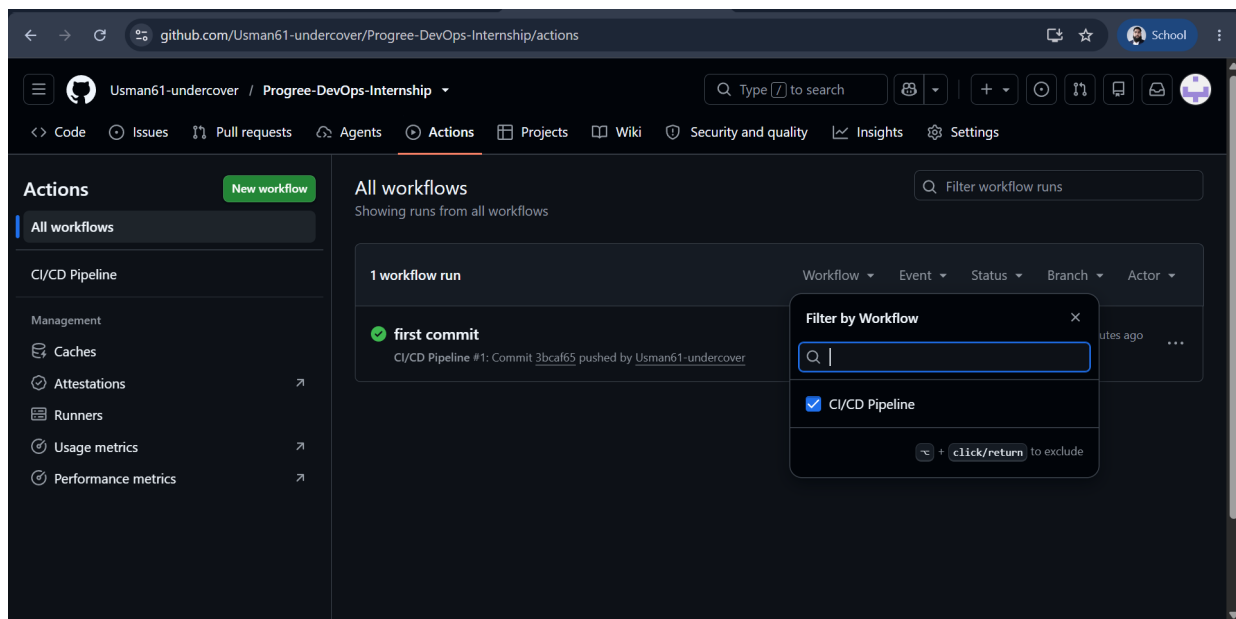


Figure 3.2 — Actions dashboard filtered by the CI/CD Pipeline workflow

3.2 Pipeline Stage Breakdown

The run summary confirms all four pipeline stages executed and passed in sequence: Lint Code (10s), Run Unit Tests (11s), Build Docker Image (24s), and Report Deployment Status (4s), for a total run duration of 59 seconds.

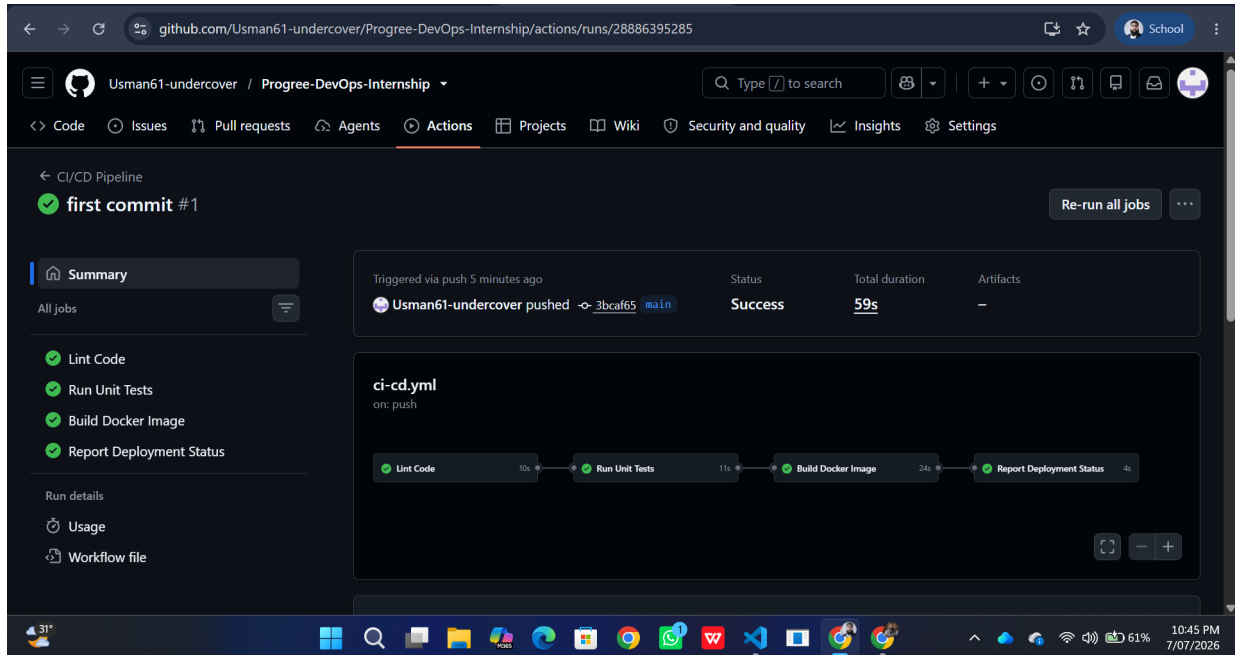


Figure 3.3 — Full pipeline visualization showing Lint Code -> Run Unit Tests -> Build Docker Image -> Report Deployment Status, all passed

Stage	Duration	Result
Lint Code	10s	Passed
Run Unit Tests	11s	Passed
Build Docker Image	24s	Passed
Report Deployment Status	4s	Passed

End of Report — Tasks 1 through 3 completed and verified with supporting evidence as shown above.